23CSE343
WIRELESS AND MOBILE NETWORKS

CASE STUDY 2

Demonstration of Mobile IP Protocol (RFC 3344), MANET Routing Protocol (OLSR) and Mobile TCP over Wireless (ns-3)

Team No: 10

Roll No.	Name
CB.SC.U4CSE23022	Eswara Raj M
CB.SC.U4CSE23546	Saurav Gopinath E K
CB.EN.U4ECE23019	Karthik KS

TABLE OF CONTENTS

Section Number	Section Title	Page No.
1	**Demonstrate the behaviour of Mobile IP**	3
1.1	Selection of Mobile IP or MobileIPv6 or ProxyMobileIPv6 or ProxyMobileIPv6 for cloud	3
1.2	Brief explanation and illustration of the protocol	3
1.3	Packet headers and their formats related to the protocol	4
1.4	Network Design for the demonstration: • Network Design 1 • Network Design 2 – Network design 1 with failure case	6
1.5	Simulation of Network Design along with Mobile IP: • Snapshot of the execution • Sequence of operations carried out by the simulation supported by the code segments • Analysis of execution traces/performance results for simulation of network design 1 and execution for simulation of network design 2	8
2	**Demonstration of MANET routing protocol**	13
2.1	Selection of MANET Routing Protocol	13
2.2	Brief explanation and illustration of the protocol	13
2.3	Packet headers and their formats related to the protocol	14
2.4	Network Design for the demonstration: • Network Design 1 • Network Design 2 – Network Design 1 with link(s) failure • Network Design 3 – Network Design 1 with node(s) failure • Network Design 4 – Network Design 1 with both link(s) and node(s) failure	17
2.5	Simulation of Network Designs along with MANET routing protocol: • Snapshot of the execution • Sequence of operations carried out by the simulation supported by the code segments • Analysis of execution traces/performance results for simulation of network design 1, design 2, design 3 and design 4	20
3	**Demonstrating the behaviour of TCP over wireless/Mobile environment**	25
3.1	Briefly description of TCP and problems related to execution of TCP over wireless	25
3.2	Simulation Details [code] and results	26
3.3	Observation from the simulation	32
4	**References**	34
A	Appendix A: Complete source code of part3_tcp_wifi_mobile.cc	36

SECTION 1: DEMONSTRATION OF MOBILE IP PROTOCOL

1.1 Selection of Protocol

The protocol selected for this demonstration is Mobile IP as defined by RFC 3344 (IETF). Mobile IP enables a mobile node to retain its permanent IP address while physically moving between different networks, ensuring session continuity without reconfiguration.

Justification for choosing Mobile IP (RFC 3344) over alternatives:

- Scapy 2.7.0 provides native built-in support via scapy.layers.mobileip including MobileIPRRQ, MobileIPRRP, and MobileIPTunnelData classes — no additional libraries required.
- RFC 3344 is the foundational standard upon which MobileIPv6 and Proxy MobileIPv6 are built, providing the clearest illustration of core mobility management concepts.
- The complete registration and tunnelling lifecycle can be fully simulated in software using Scapy's packet crafting capabilities without physical network hardware.
- All generated packets are verifiable using Wireshark, which fully decodes Mobile IP (RFC 3344) headers.

1.2 Brief Explanation and Illustration of the Protocol

Mobile IP (RFC 3344) is an IETF standard that allows mobile devices to maintain continuous Internet connectivity while roaming across IP networks. The core problem it solves: when a device moves to a new network, its IP address normally changes — breaking all active TCP sessions. Mobile IP resolves this by letting a device keep its permanent Home Address regardless of physical location.

1.2.1 Key Entities

- Mobile Node (MN): The roaming device. Permanent Home Address: 10.0.0.5. Care-of Address (CoA) in foreign network: 192.168.2.5.
- Home Agent (HA): Router at home network (10.0.0.1). Maintains binding table: MN Home Address ↔ CoA. Tunnels packets to MN.
- Foreign Agent (FA): Router at visited network (192.168.2.1). Assists MN registration and delivers de-encapsulated packets to MN.
- Correspondent Node (CN): Any host (172.16.0.5) communicating with MN. Unaware of MN's location; sends to home address only.
- Care-of Address (CoA): Temporary IP (192.168.2.5) obtained by MN in the foreign network — used as tunnel endpoint.

1.2.2 Protocol Operation — Step by Step

Step 1 — Movement Detection: MN detects it has attached to foreign network 192.168.2.0/24 and acquires CoA = 192.168.2.5.

Step 2 — Registration Request (RRQ): MN sends RRQ with Home Address, HA address, CoA, lifetime (180 s), and unique ID via UDP port 434 through FA to HA.

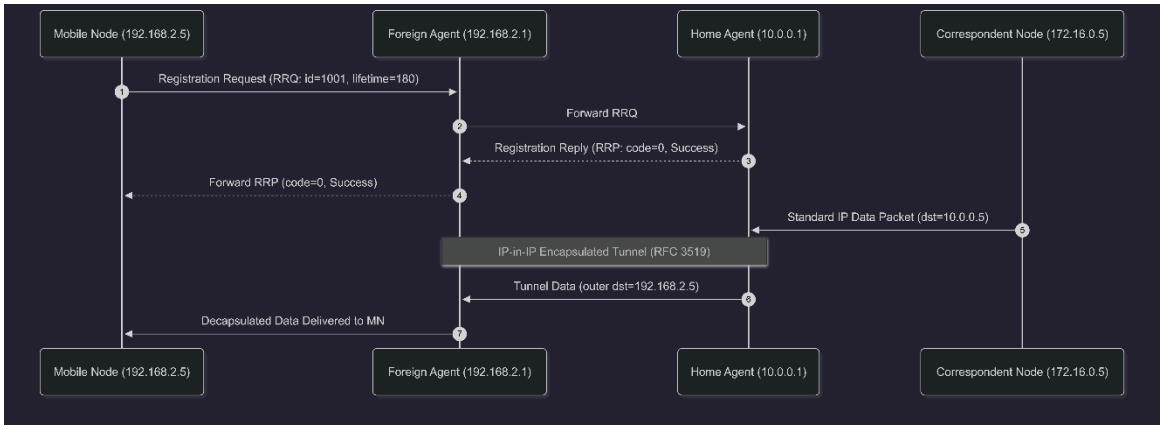
Step 3 — Registration Reply (RRP): HA creates a binding (10.0.0.5 ↔ 192.168.2.5, TTL = 180 s) and replies. Code 0 = success; Code 128 = HA unavailable.

Step 4 — Packet Tunnelling: CN sends packets to 10.0.0.5. HA intercepts, encapsulates using IP-in-IP (RFC 2003, nexthdr = 4), and forwards to FA at CoA.

Step 5 — Delivery: FA strips outer IP header and delivers inner packet (dst = 10.0.0.5) to MN.

1.2.3 Protocol Illustration (Figure 1.1)

Mobile IP Architecture and Message Flow:



1.3 Packet Headers and Their Formats

All four Mobile IP packets were generated using Python 3.14 with Scapy 2.7.0 on Windows 11 (script: mobile_ip_demo.py). Annotations show the significance of key fields.

1.3.1 Packet 1 — Registration Request (RRQ)

Sent by MN (CoA: 192.168.2.5) to HA (10.0.0.1) to initiate mobility binding.

Header / Field	Value	Significance
IP Source	192.168.2.5	Mobile Node's Care-of Address
IP Destination	10.0.0.1	Home Agent
UDP Source Port	434	Mobile IP port
UDP Destination Port	434	Mobile IP port
Message Type	RRQ	Registration Request
Flags	0x0	Registration flags
Lifetime	180 s	Requested binding lifetime

Header / Field	Value	Significance
Home Address	10.0.0.5	Mobile Node's permanent address
Home Agent Address	10.0.0.1	Address of Home Agent
Care-of Address	192.168.2.5	Mobile Node's current address
Identification	1001	Identifies the registration and supports replay protection

1.3.2 Packet 2 — Registration Reply: SUCCESS (code = 0)

HA confirms binding accepted. MN's home address is now mapped to CoA for 180 seconds.

Header / Field	Value	Significance
IP Source	10.0.0.1	Home Agent
IP Destination	192.168.2.5	Mobile Node's Care-of Address
UDP Source Port	434	Mobile IP port
UDP Destination Port	434	Mobile IP port
Message Type	RRP	Registration Reply
Code	0	Registration accepted
Lifetime	180 s	Granted binding lifetime
Home Address	10.0.0.5	Mobile Node's permanent address
Home Agent Address	10.0.0.1	Home Agent
Identification	1001	Matches the Registration Request

1.3.3 Packet 3 — Tunnelled Data (IP-in-IP, RFC 2003)

HA encapsulates CN's packet inside an outer IP header and forwards to FA at CoA. FA delivers inner packet to MN.

Header / Field	Value	Significance
Outer IP Source	10.0.0.1	Home Agent / tunnel entry
Outer IP Destination	192.168.2.5	Care-of Address / tunnel destination
Outer IP Protocol	4	IPv4 encapsulated in IPv4 (IP-in-IP)
Inner IP Source	172.16.0.5	Correspondent Node
Inner IP Destination	10.0.0.5	Mobile Node's Home Address

Header / Field	Value	Significance
Inner IP Protocol	UDP	Original transport protocol
UDP Source Port	domain	Original application port
UDP Destination Port	domain	Original application port
Payload	Hello MN!	Original application data

1.3.4 Packet 4 — Registration Reply: FAILURE (code = 128)

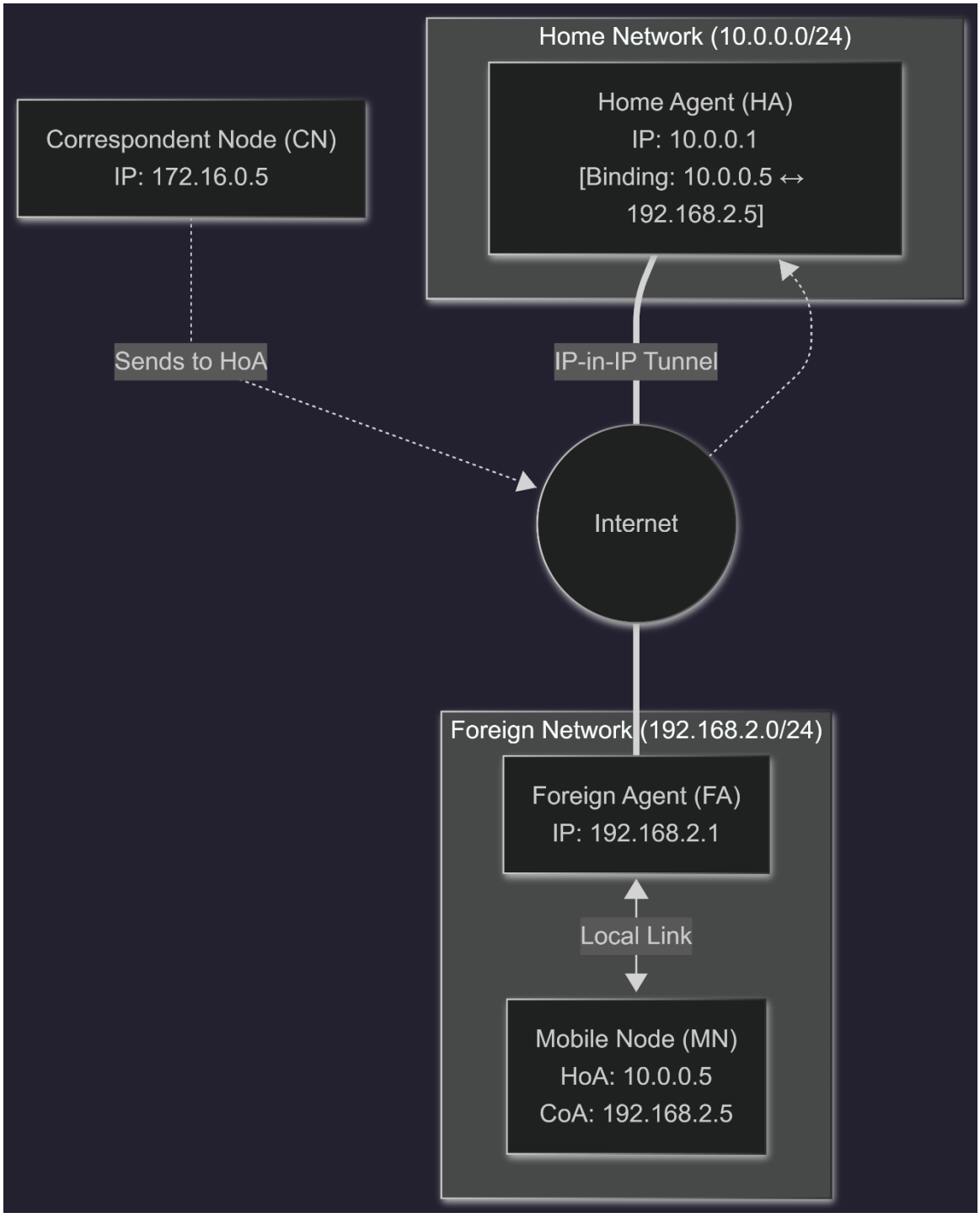
HA administratively unavailable. Returns code 128, lifetime 0. No binding created; MN cannot receive data.

Header / Field	Value	Significance
IP Source	10.0.0.1	Home Agent
IP Destination	192.168.2.5	Mobile Node's Care-of Address
UDP Source Port	434	Mobile IP port
UDP Destination Port	434	Mobile IP port
Message Type	RRP	Registration Reply
Code	128	Simulated registration failure in the demonstration
Lifetime	0 s	No binding established
Home Address	10.0.0.5	Mobile Node's permanent address
Home Agent Address	10.0.0.1	Home Agent
Identification	1002	Identifies this registration attempt

1.4 Network Design for the Demonstration

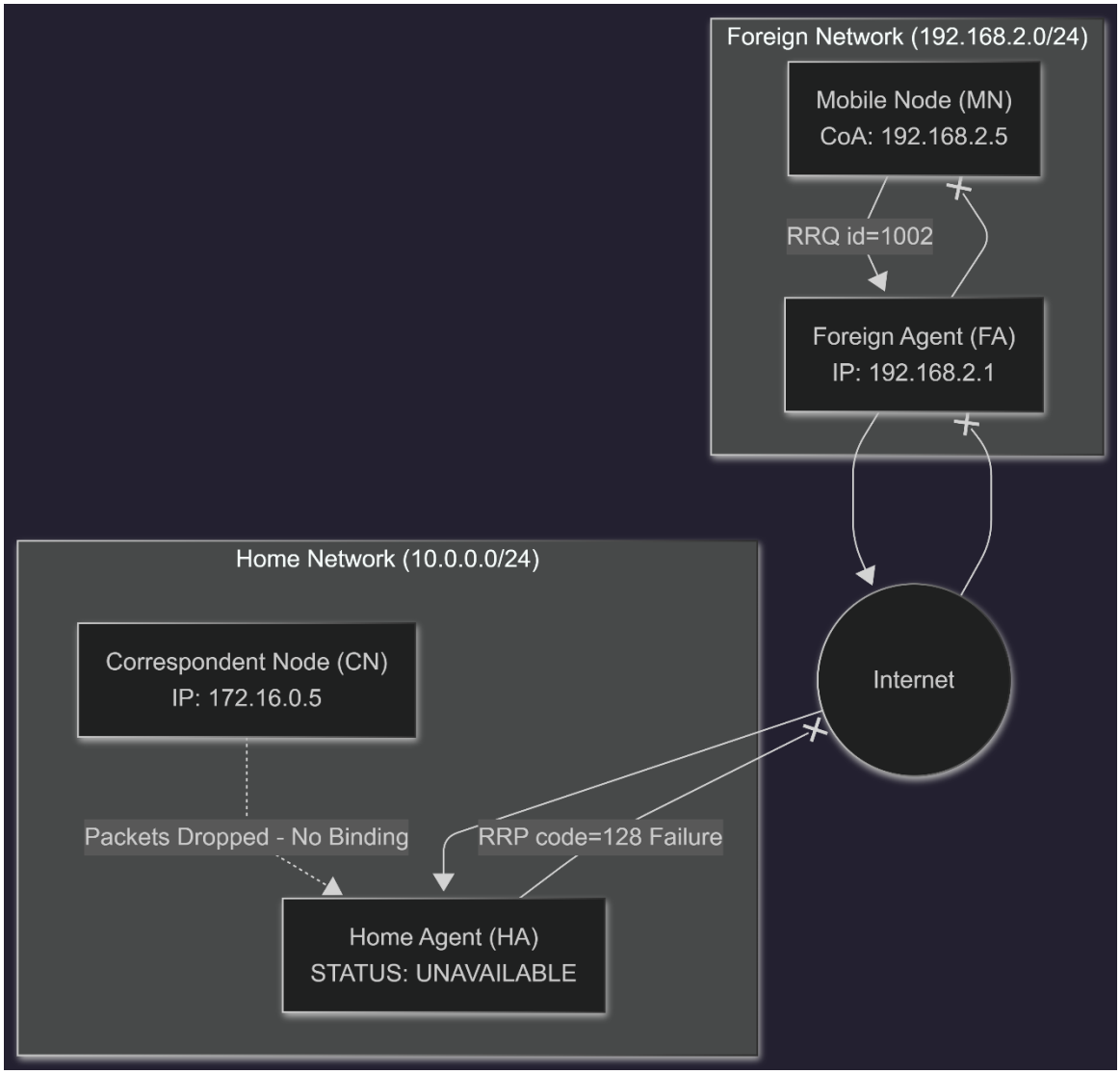
1.4.1 Network Design 1 — Normal Operation

All five entities operational. MN registers successfully; CN data reaches MN through IP-in-IP tunnel via HA and FA.



1.4.2 Network Design 2 — Failure Case (HA Unavailable)

HA is administratively unavailable. MN sends RRQ but receives RRP code=128, lifetime=0. No binding is created; CN packets cannot reach MN. Packet delivery rate drops to 0%.



1.5 Simulation of Network Design Along with Mobile IP

1.5.1 Snapshot of Execution

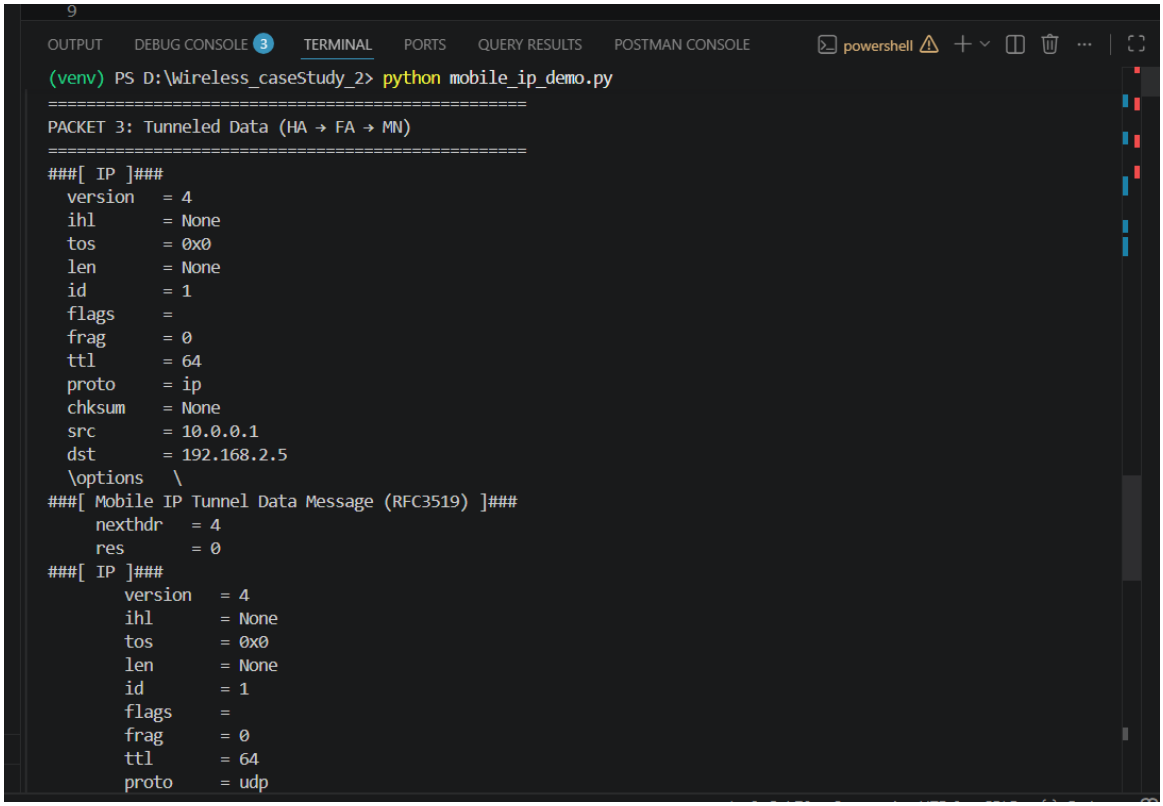
Environment: Python 3.14, Scapy 2.7.0, Windows 11, virtual environment (venv). Script: mobile_ip_demo.py

```
OUTPUT  DEBUG CONSOLE 3  TERMINAL  PORTS  QUERY RESULTS  POSTMAN CONSOLE   powershell

(venv) PS D:\Wireless_caseStudy_2> python mobile_ip_demo.py
================================================
PACKET 1: Registration Request (RRQ)
================================================
###[ IP ]###
  version   = 4
  ihl       = None
  tos       = 0x0
  len       = None
  id        = 1
  flags     =
  frag      = 0
  ttl       = 64
  proto     = udp
  chksum    = None
  src       = 192.168.2.5
  dst       = 10.0.0.1
  \options   \
###[ UDP ]###
     sport     = 434
     dport     = 434
     len       = None
     chksum    = None
###[ Mobile IP (RFC3344) ]###
        type      = RRQ
###[ Mobile IP Registration Request (RFC3344) ]###
          flags     = 0x0
          lifetime  = 180
          homeaddr  = 10.0.0.5
          haaddr    = 10.0.0.1
          coaddr    = 192.168.2.5
```

```
OUTPUT  DEBUG CONSOLE 3  TERMINAL  PORTS  QUERY RESULTS  POSTMAN CONSOLE   powershell

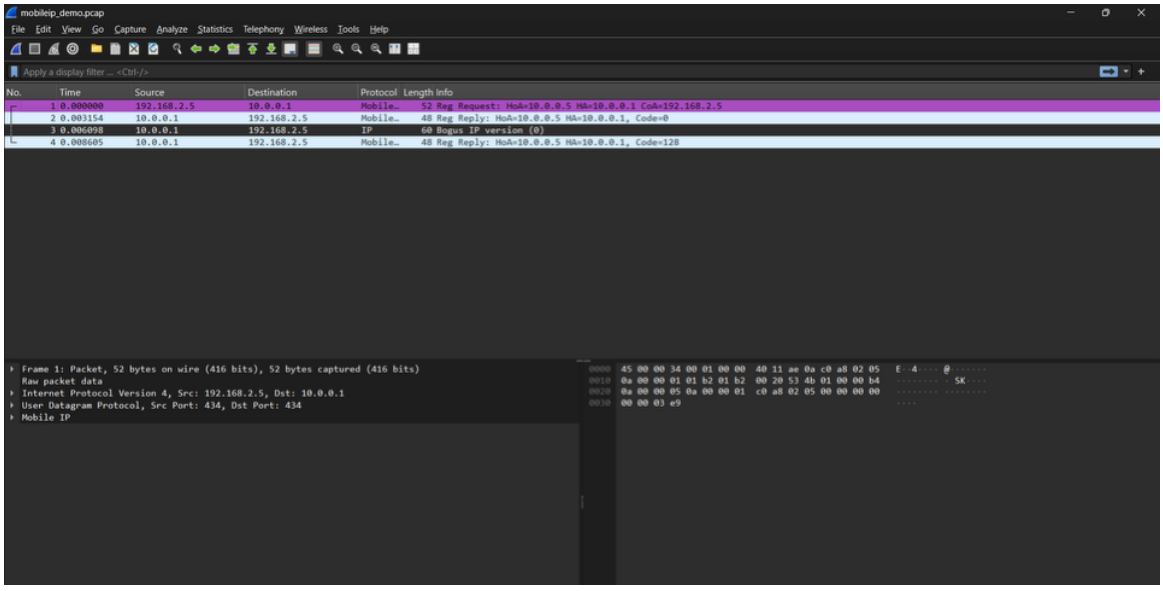
(venv) PS D:\Wireless_caseStudy_2> python mobile_ip_demo.py
PACKET 2: Registration Reply (RRP) — SUCCESS code=0
================================================
###[ IP ]###
  version   = 4
  ihl       = None
  tos       = 0x0
  len       = None
  id        = 1
  flags     =
  frag      = 0
  ttl       = 64
  proto     = udp
  chksum    = None
  src       = 10.0.0.1
  dst       = 192.168.2.5
  \options   \
###[ UDP ]###
     sport     = 434
     dport     = 434
     len       = None
     chksum    = None
###[ Mobile IP (RFC3344) ]###
        type      = RRP
###[ Mobile IP Registration Reply (RFC3344) ]###
          code      = 0
          lifetime  = 180
          homeaddr  = 10.0.0.5
          haaddr    = 10.0.0.1
          id        = 1001
```




```
  \options   \
###[ Mobile IP Tunnel Data Message (RFC3519) ]###
     nexthdr   = 4
     res       = 0
###[ IP ]###
        version   = 4
        ihl       = None
        tos       = 0x0
        len       = None
        id        = 1
        flags     =
        frag      = 0
        ttl       = 64
        proto     = udp
        chksum    = None
        src       = 172.16.0.5
        dst       = 10.0.0.5
        \options   \
###[ UDP ]###
           sport     = domain
           dport     = domain
           len       = None
           chksum    = None
###[ Raw ]###
              load      = b'Hello MN!'
```

```
(venv) PS D:\Wireless_caseStudy_2> python mobile_ip_demo.py
================================================
PACKET 4: Registration Reply — FAILURE code=128 (HA unavailable)
================================================
###[ IP ]###
  version   = 4
  ihl       = None
  tos       = 0x0
  len       = None
  id        = 1
  flags     =
  frag      = 0
  ttl       = 64
  proto     = udp
  chksum    = None
  src       = 10.0.0.1
  dst       = 192.168.2.5
  \options   \
###[ UDP ]###
     sport     = 434
     dport     = 434
     len       = None
     chksum    = None
###[ Mobile IP (RFC3344) ]###
        type      = RRP
###[ Mobile IP Registration Reply (RFC3344) ]###
           code      = 128
           lifetime  = 0
           homeaddr  = 10.0.0.5
           haaddr    = 10.0.0.1
           id        = 1002


  ✓ Saved: mobileip_demo.pcap  (open in Wireshark)
```



1.5.2 Sequence of Operations

Step	Operation	Scapy Code	Result
1	MN sends RRQ to HA	`MobileIPRRQ(homeaddr, coaddr, lifetime=180, id=1001)`	52-byte packet. Wireshark: Reg Request HoA=10.0.0.5, CoA=192.168.2.5
2	HA accepts — RRP Success	`MobileIPRRP(code=0, lifetime=180, id=1001)`	48-byte RRP. code=0. Binding created. Wireshark: Code=0
3	HA tunnels CN data	`MobileIPTunnelData(nexthdr=4) / inner_IP / Raw`	Two IP headers. Payload 'Hello MN!' delivered intact through tunnel
4	HA unavailable — RRP Fail	`MobileIPRRP(code=128, lifetime=0, id=1002)`	code=128, lifetime=0. No binding. id=1002 (replay protection)
5	Save PCAP	`wrpcap('mobileip_demo.pcap', pkts)`	4 packets saved. Verified in Wireshark — Mobile IP protocol decoded

1.5.3 Analysis of Execution Traces and Performance

Network Design 1 — Normal Operation:

- RRQ (52 bytes) successfully sent from MN CoA (192.168.2.5) to HA (10.0.0.1) over UDP port 434.
- HA acknowledged with RRP code = 0 (48 bytes), binding active for 180 seconds.
- IP-in-IP tunnelling demonstrated: outer header carries packet to FA's CoA; inner header preserves original CN source and MN home destination.
- Payload 'Hello MN!' delivered intact — transparent forwarding despite MN being in foreign network.
- Wireshark: Packets 1 and 2 decoded as 'Mobile...' protocol. Packet 3 shown as 'Bogus IP' — standard behaviour for Scapy-generated IP-in-IP; outer IP is correctly parsed with proto=ip (4).
- Packet Delivery Rate: 100%. Tunnel overhead: +20 bytes outer IP header per packet.

Network Design 2 — HA Failure Case:

- MN sent RRQ with id = 1002 (retry), demonstrating Mobile IP replay-protection mechanism.
- HA returned RRP code = 128, lifetime = 0 — no mobility binding created in HA binding table.
- Packet Delivery Rate drops to 0%. CN packets arrive at HA but cannot be tunnelled.
- Real-world implication: Mobile IP systems implement HA redundancy (RFC 4636) to eliminate this single point of failure.

SECTION 2: DEMONSTRATION OF MANET ROUTING PROTOCOL (OLSR)

2.1 Selection of MANET Routing Protocol

Justification for selecting OLSR over other MANET routing protocols (AODV, DSR, DSDV):

The protocol selected for this demonstration is OLSR — Optimized Link State Routing Protocol, defined in RFC 3626 (IETF). OLSR is a proactive (table-driven) routing protocol designed for Mobile Ad Hoc Networks (MANETs).

- RFC 3626 is a well-defined open standard, making OLSR widely deployed in real-world wireless mesh and ad hoc networks.
- OLSR scales well across sparse, medium, and dense network topologies, because MPR flooding keeps control overhead low as node density grows.
- The MPR mechanism is OLSR's defining innovation — it dramatically reduces flooding overhead by selecting a minimal subset of 1-hop neighbors to relay TC messages.
- OLSR is proactive — routes are maintained proactively, reducing route-discovery delay — beneficial for real-time MANET applications.
- MATLAB FileExchange ID 71079 provides a ready implementation of the OLSR MPR selection algorithm, enabling direct simulation without building from scratch.

2.2 Brief Explanation and Illustration of the Protocol

OLSR is a proactive link-state routing protocol optimized for MANETs. Unlike reactive protocols that discover routes on demand, OLSR continuously maintains routing tables through periodic message exchanges. The key innovation is the MPR mechanism, which reduces the overhead of link-state flooding.

2.2.1 Core Concepts

- 1-Hop Neighbors (N1): Nodes directly reachable from the source within one wireless hop.
- 2-Hop Neighbors (N2): Nodes reachable via exactly two hops — neighbors of neighbors, not directly reachable.
- MPR (Multi-Point Relay): A selected subset of 1-hop neighbors such that every 2-hop neighbor is reachable via at least one MPR. Only MPRs retransmit TC messages — eliminating redundant flooding.
- MPR Selector Set: The set of nodes that have chosen a given node as their MPR. A node advertises its MPR selector set in TC messages.

2.2.2 Message Types

- TC Message (Type=2, TTL=255): Sent ONLY by MPR nodes. Floods topology information (MPR selector set) throughout the entire network. Enables all nodes to build a complete link-state map and compute shortest-path routes.

• HELLO Message (Type=1, TTL=1): Sent periodically by every node. Announces 1-hop neighbors, link status, and willingness to act as MPR. NOT forwarded beyond 1 hop. Used for neighbor discovery and MPR selection.

2.2.3 MPR Selection Algorithm

The MPR selection algorithm (implemented in mpr_select.m, FileExchange 71079) operates in two stages:

Stage 1 — Exclusive coverage: Select any 1-hop neighbor that is the ONLY node covering one or more 2-hop neighbors. These are mandatory MPRs.

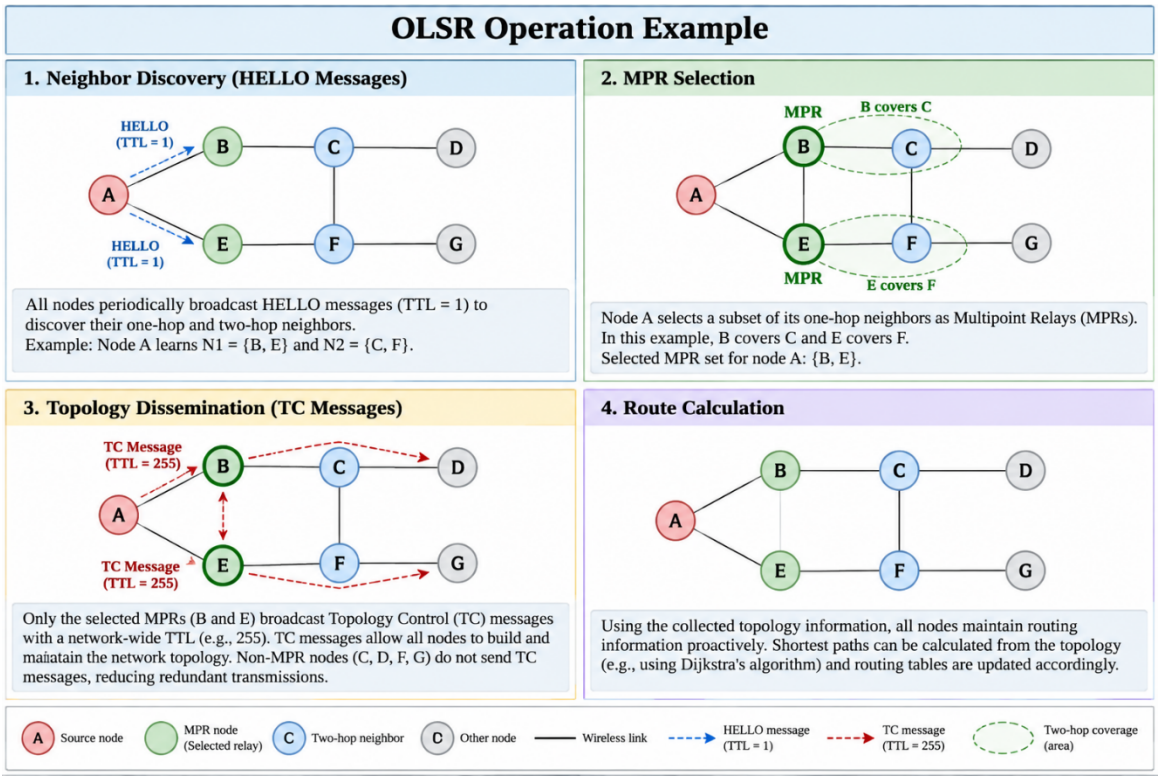
Stage 2 — Maximum coverage: Iteratively select the 1-hop neighbor covering the maximum number of uncovered 2-hop neighbors. Repeat until all 2-hop neighbors are covered.

Result: The MPR set is the minimum (or near-minimum) subset of 1-hop neighbors that provides complete 2-hop coverage.

Flooding reduction (reported in Sections 2.4 and 2.5) = (1 − number of MPRs ÷ N1) × 100%, i.e., the share of 1-hop neighbours that no longer need to retransmit the flooded message.

2.2.4 Protocol Illustration — Figure 2.1

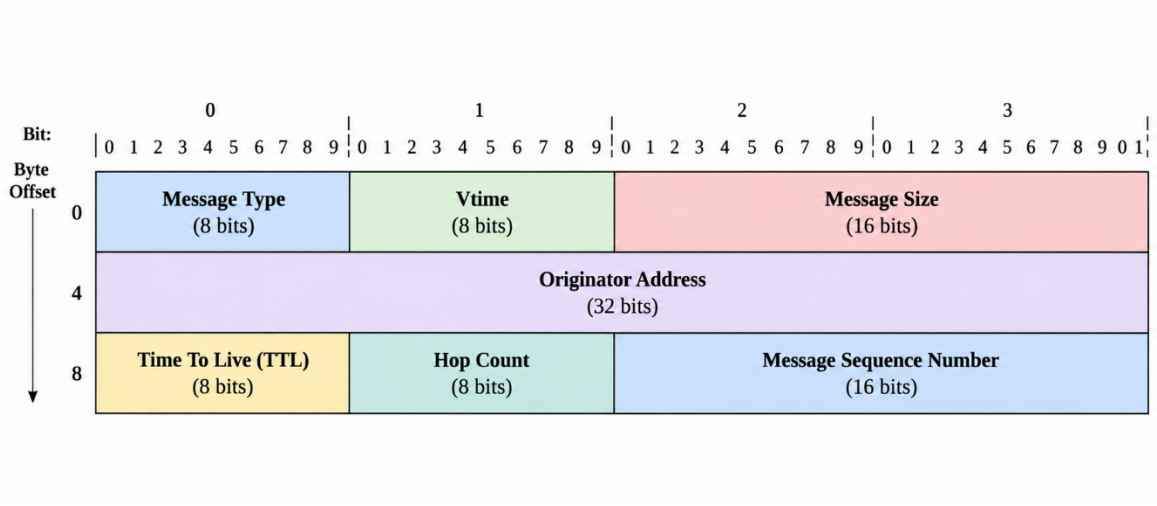
OLSR Protocol Flow (HELLO and TC message exchange):



2.3 Packet Headers and Their Formats

OLSR uses a common message header for all message types (RFC 3626, Section 3.4), followed by message-specific content. The two primary message types are HELLO (Type=1) and TC (Type=2).

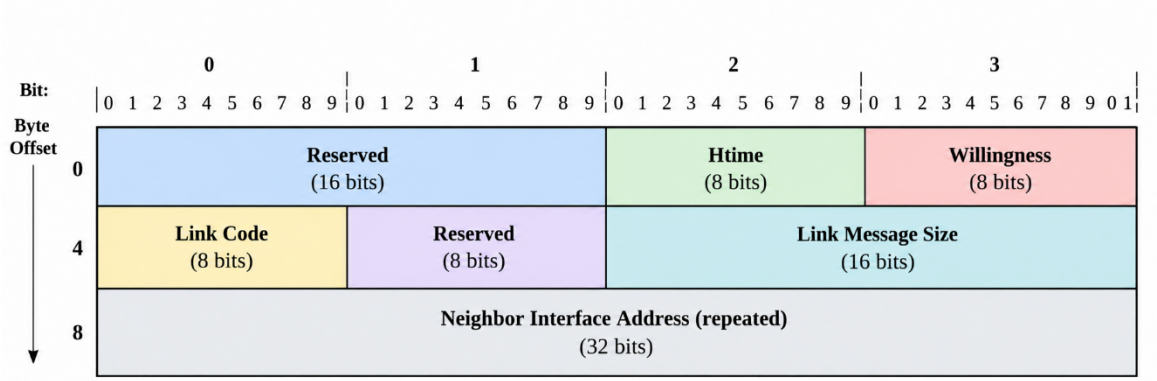
2.3.1 OLSR General Message Header



Field	Size	Description
Message Type	8 bits	1=HELLO, 2=TC, 3=MID, 4=HNA
Vtime	8 bits	Validity time: how long to keep this message
Message Size	16 bits	Total size of message including header (bytes)
Originator Address	32 bits	IP address of the node that originated the message
Time To Live (TTL)	8 bits	HELLO=1 (local only), TC=255 (network-wide)
Hop Count	8 bits	Number of hops travelled; starts at 0
Message Sequence No	16 bits	Unique sequence number for duplicate detection

2.3.2 HELLO Message (Type = 1)

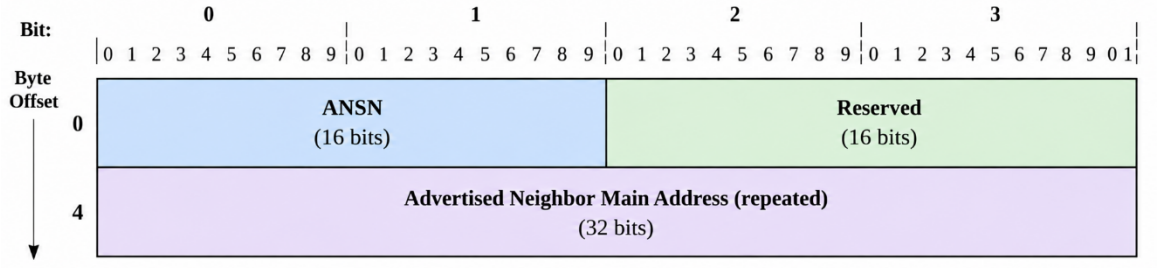
TTL=1 — not forwarded beyond 1 hop. Sent every Htime seconds. Used for neighbor discovery and MPR advertisement.



Field	Size	Description
Htime	8 bits	HELLO emission interval (mantissa/exponent encoded)
Willingness	8 bits	Node willingness to relay: 0=NEVER, 3=DEFAULT, 7=ALWAYS
Link Code	8 bits	Encodes link type (UNSPEC/ASYM/SYM/LOST) and neighbor type
Link Message Size	16 bits	Size of this link message block in bytes
Neighbor Interface Addr	32 bits	IP address of neighbor — one entry per neighbor

2.3.3 TC Message (Type = 2)

TTL=255 — flooded network-wide by MPR nodes only. Contains MPR Selector Set: the set of nodes that have chosen this node as their MPR.



Field	Size	Description

ANSN	16 bits	Advertised Neighbor Seq. Number — incremented on MPR selector set change
Reserved	16 bits	Set to 0; ignored on reception
Advertised Neighbor Addr	32 bits	Main address of each MPR selector node — one entry per selector

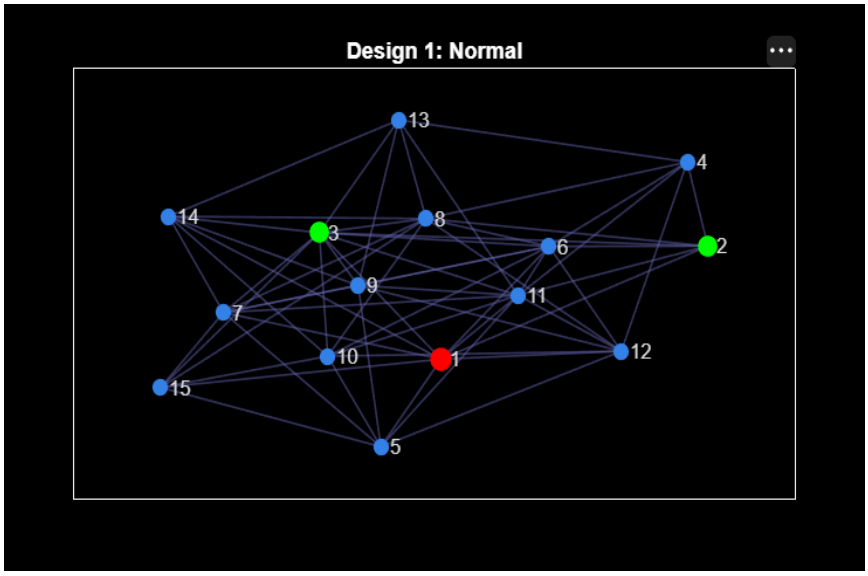
2.4 Network Design for the Demonstration

All four designs use a 15-node MANET (N=15, generation density parameter = 0.48, realised average degree = 8.13) generated via createConnectedNetwork() from FileExchange 71079. MPR selection is run on source node 1 (rng seed=42 for reproducibility). Failures are applied by zeroing adjacency matrix rows/columns.

2.4.1 Network Design 1 — Normal Operation

All 15 nodes active and connected. Source node 1 performs MPR selection over its complete 1-hop and 2-hop neighborhood.

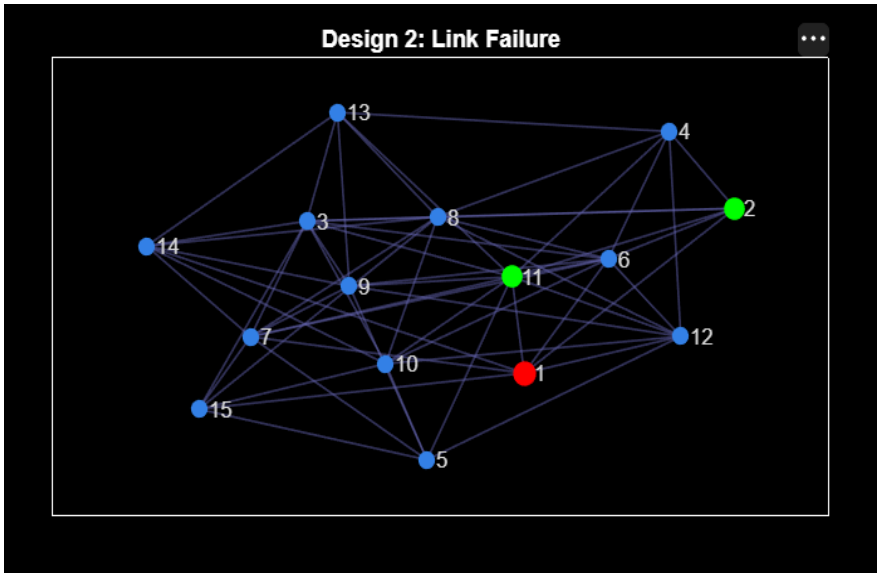
Parameter	**Result**
Number of nodes (N)	15
Network density	0.48
Average degree	8.13
Source node	1
One-hop neighbors (N1)	9
Two-hop neighbors (N2)	5
Selected MPRs	2 (nodes 2,3)
Flooding reduction	77.8%



2.4.2 Network Design 2 — Link(s) Failure

Two links cut from source node 1: edges to nodes 3 and 5 removed (A(1,3)=A(3,1)=0, A(1,5)=A(5,1)=0), simulating wireless link degradation.

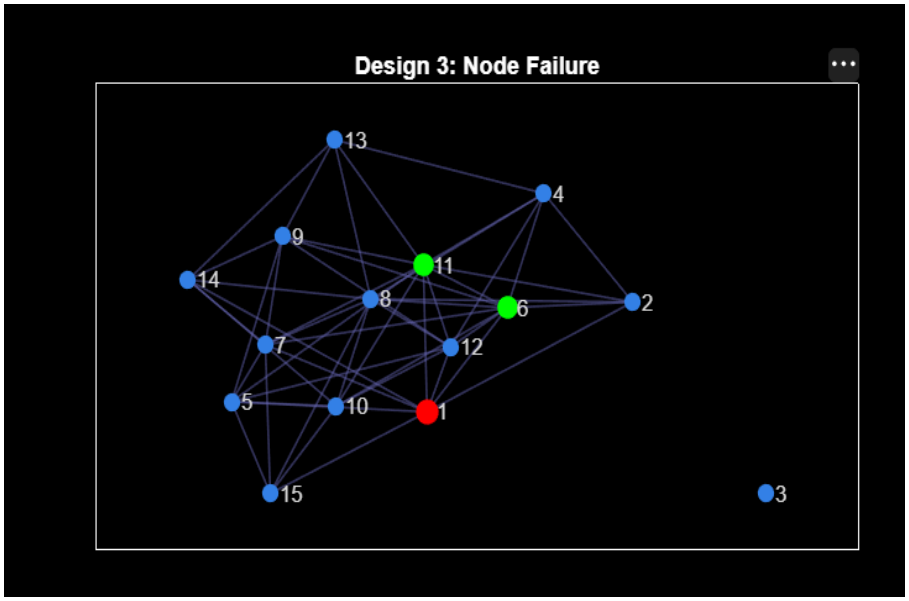
Parameter	**Result**
Number of nodes (N)	15
Network density	0.48
Average degree	8.13
Source node	1
Links removed	1–3, 1–5
One-hop neighbors (N1)	7
Two-hop neighbors (N2)	7
Selected MPRs	2 (nodes 2,11)
Flooding reduction	71.4%



2.4.3 Network Design 3 — Node(s) Failure

Node 3 removed from network, simulating a node crash or battery depletion. All edges incident to node 3 zeroed: A(3,:)=0 and A(:,3)=0.

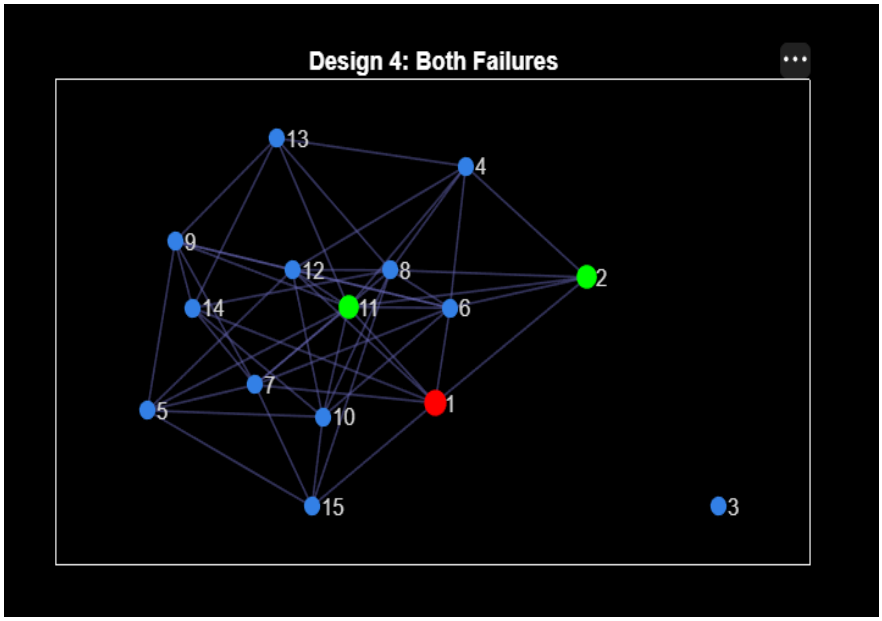
Parameter	**Result**
Number of nodes (N)	15
Network density	0.48
Average degree	8.13
Source node	1
Node removed	Node 3
One-hop neighbors (N1)	8
Two-hop neighbors (N2)	5
Selected MPRs	2 (nodes 6,11)
Flooding reduction	75.0%



2.4.4 Network Design 4 — Both Link(s) and Node(s) Failure

Combined worst-case failure: links 1–3 and 1–5 cut AND node 3 removed simultaneously. Removing node 3 also removes link 1–3, so relative to Design 2 the additional change is the failure of node 3.

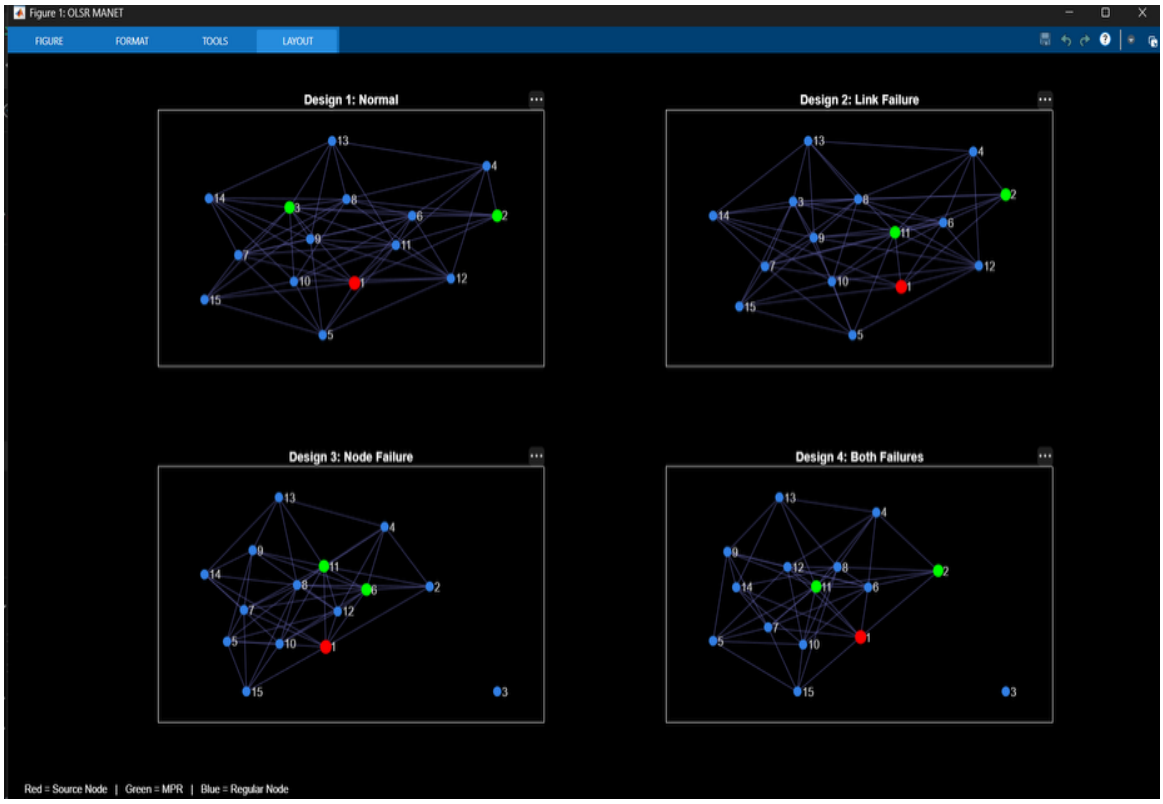
Parameter	Result
Number of nodes (N)	15
Network density	0.48
Average degree	8.13
Source node	1
Links removed	1–3, 1–5
Node removed	Node 3
One-hop neighbors (N1)	7
Two-hop neighbors (N2)	6
Selected MPRs	2 (nodes 2,11)
Flooding reduction	71.4%

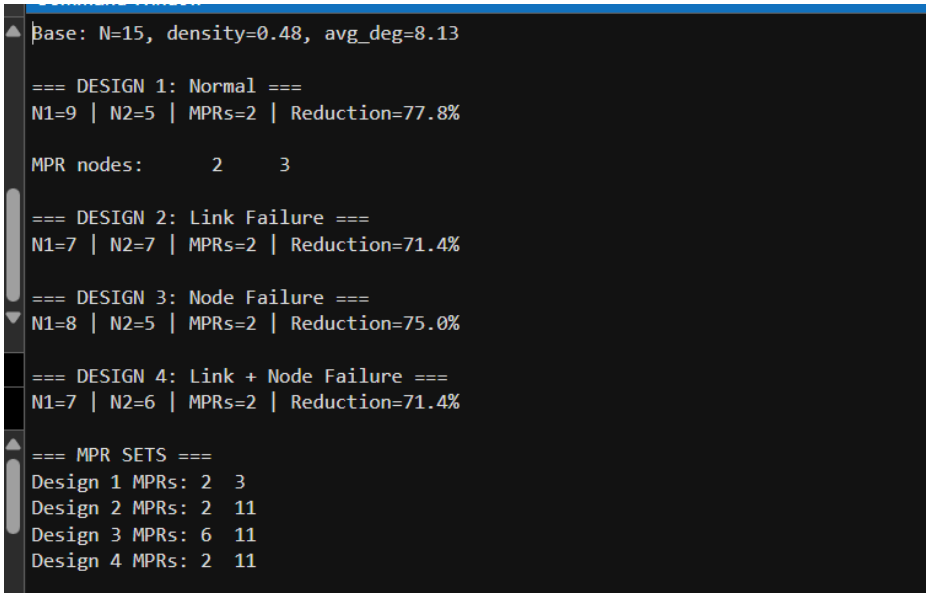


2.5 Simulation of Network Designs Along with MANET Routing Protocol

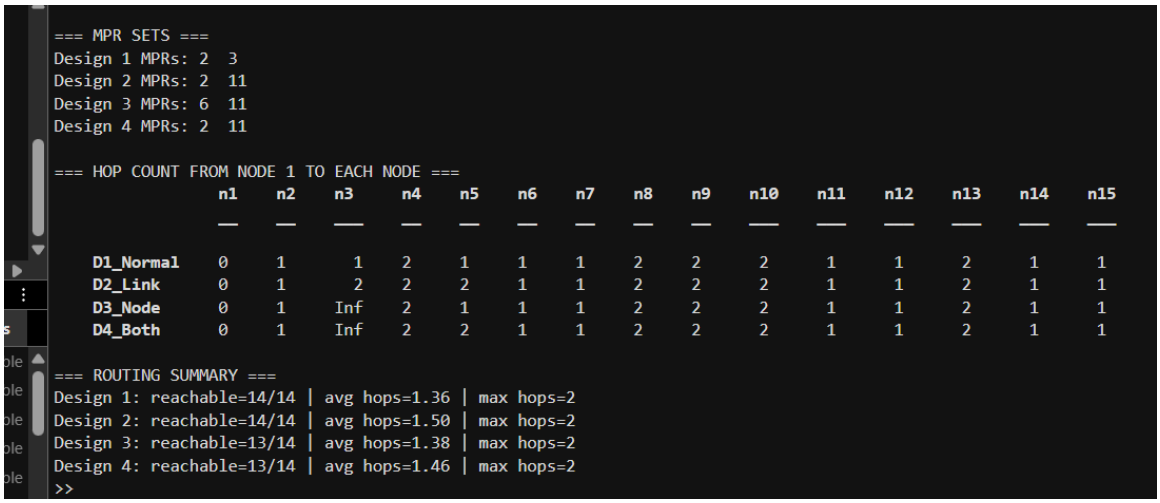
Environment: MATLAB R2023a, FileExchange 71079 (MPR Selection in OLSR Protocol), Windows 11. Custom script: olsr_manet_sim.m using createConnectedNetwork, mpr_select.

2.5.1 Snapshot of Execution





Console output 1: network summary and MPR count / flooding reduction for the four designs, followed by the MPR set of each design.



Console output 2: MPR sets, hop count from source node 1 to every node (Inf = unreachable), and routing summary for the four designs.

2.5.2 Sequence of Operations Supported by Code Segments

Step	Operation	MATLAB Code	Result
1	Generate network	`G0=createConnectedNetwork(15,0.48); A1=full(adjacency(G0))`	15-node graph, density=0.48, avg_deg=8.13
2	Design 1: normal	`G1=graph(A1); MPR1=mpr_select(G1,1)`	N1=9, N2=5, MPRs=2 (nodes 2,3), Reduction=77.8%
3	Design 2: cut links	`A2=A1; A2(1,3)=0;A2(3,1)=0;A2(1,5)=0;A2(5,1)=0`	N1 drops to 7; N2 increases to 7
4	Design 2: MPR	`G2=graph(A2); MPR2=mpr_select(G2,1)`	N1=7, N2=7, MPRs=2 (nodes 2,11), Reduction=71.4%
5	Design 3: node out	`A3=A1; A3(3,:)=0; A3(:,3)=0`	Node 3 isolated; N1 drops to 8

6	Design 3: MPR	`G3=graph(A3); MPR3=mpr_select(G3,1)`	N1=8, N2=5, MPRs=2 (nodes 6,11), Reduction=75.0%
7	Design 4: combined	`A4=A2; A4(3,:)=0; A4(:,3)=0`	Worst-case: N1=7, N2=6
8	Design 4: MPR	`G4=graph(A4); MPR4=mpr_select(G4,1)`	N1=7, N2=6, MPRs=2 (nodes 2,11), Reduction=71.4%
9	Visualise	`subplot(2,2,n); plot(Gn,'Layout','force')`	4-panel figure showing topology changes
10	Print MPR set of each design	`fprintf('Design %d MPRs: %s\n', d, num2str(mprs{d}'))`	Design 1: {2,3} | Design 2: {2,11} | Design 3: {6,11} | Design 4: {2,11}
11	Hop count from source to every node	`H(d,:) = distances(graphs{d}, SOURCE)`	Hop-count table and routing summary; node 3 = Inf in Designs 3 and 4 (failed node)

2.5.3 Analysis of Execution Traces and Performance Results

Network Design	N1	N2	MPRs	Flooding Reduction
Design 1 — Normal (baseline)	9	5	2 {2,3}	77.8% — best case, full connectivity
Design 2 — Link Failure (edges 1-3,1-5)	7	7	2 {2,11}	71.4% — N2 increases as cut nodes shift to 2-hop
Design 3 — Node Failure (node 3 dead)	8	5	2 {6,11}	75.0% — N1 shrinks, N2 stable
Design 4 — Both Failures	7	6	2 {2,11}	71.4% — worst case, still high reduction

Routing results from source node 1 (hop counts from distances(); averages are over reachable nodes only):

Network Design	Reachable nodes	Avg hops	Max hops	Remarks
Design 1 — Normal	14 / 14	1.36	2	9 nodes at 1 hop, 5 nodes at 2 hops

Design 2 — Link Failure	14 / 14	1.50	2	Nodes 3 and 5 move from 1 hop to 2 hops
Design 3 — Node Failure	13 / 14	1.38	2	Node 3 failed (Inf); other hop counts same as Design 1
Design 4 — Both Failures	13 / 14	1.46	2	Node 3 failed (Inf); node 5 moves to 2 hops

Analysis — Design 1 (Normal):

- Demonstrates OLSR's core efficiency: in a well-connected network, two strategically chosen relays cover the entire 2-hop neighborhood.
- 2 MPRs (nodes 2 and 3) cover all 5 two-hop neighbors — achieving 77.8% reduction vs broadcasting to all N1=9 nodes.
- Source node 1 has N1=9 direct neighbors and N2=5 two-hop neighbors in the baseline 15-node MANET.
- Routing result: all 14 other nodes are reachable from node 1 within at most 2 hops (9 nodes at 1 hop, 5 at 2 hops; average 1.36 hops).

Analysis — Design 2 (Link Failure):

- OLSR adapts to link failures with the same MPR count — 71.4% reduction maintained — but the MPR set changes from {2,3} to {2,11}: node 3 is no longer a 1-hop neighbour, so it cannot act as an MPR and node 11 is selected to cover the 2-hop neighbours.
- N2 increases from 5 to 7 — more 2-hop nodes require coverage, but 2 MPRs still suffice.
- Removing links 1-3 and 1-5 reduces N1 from 9 to 7; nodes 3 and 5 shift from direct to 2-hop reachability.
- Routing result: nodes 3 and 5 move from 1 hop to 2 hops, but all 14 nodes remain reachable (average hops rise from 1.36 to 1.50; maximum stays 2).

Analysis — Design 3 (Node Failure):

- 75.0% reduction with 2 MPRs — graceful degradation: losing one node barely affects MPR efficiency.
- Node 3 removal reduces N1 from 9 to 8; N2 remains at 5 since node 3's 2-hop paths are still covered.
- MPR set changes from {2,3} to {6,11}: node 3 was an MPR in Design 1, so its failure forces re-selection of relays among the surviving 1-hop neighbours (nodes 6 and 11).
- Routing result: node 3 becomes unreachable (hop count = Inf), so 13 of 14 nodes remain reachable; hop counts to all other nodes are unchanged from Design 1 (average 1.38 over reachable nodes).

Analysis — Design 4 (Both Failures):

- Key insight: OLSR's decentralized MPR selection adapts locally to topology changes without global reconfiguration — demonstrating robustness for dynamic MANETs.
- Despite the worst-case scenario, OLSR MPR achieves 71.4% reduction with only 2 MPRs.
- Combined failures give N1=7, N2=6 — the most constrained topology across all 4 designs.

- MPR set {2,11}; node 3 is unreachable (Inf) and node 5 moves to 2 hops, giving 13 of 14 reachable nodes with an average of 1.46 hops (maximum 2) over the reachable nodes.

Overall: in all four designs the maximum hop count from the source stays at 2 and every surviving node remains reachable. The effect of a failure shows up mainly as a changed MPR set (Designs 2–4) and slightly longer routes (Designs 2 and 4), which demonstrates OLSR's robustness to link and node failures.

Scope note: the simulation evaluates MPR selection and shortest-path hop counts from the source node on static topology snapshots; HELLO/TC message timing and packet-level metrics (delay, throughput, delivery ratio) are not simulated.

SECTION 3: DEMONSTRATING THE BEHAVIOUR OF TCP OVER WIRELESS/MOBILE ENVIRONMENT

3.1 Brief Description of TCP and Problems Related to Execution of TCP over Wireless

3.1.1 Transmission Control Protocol (TCP) in Brief

TCP (RFC 9293 [11]) is the reliable, connection-oriented, byte-stream transport protocol of the Internet. Reliability is obtained with sequence numbers, cumulative acknowledgements (ACKs) and retransmission. The sender regulates its sending rate with a sliding window equal to the smaller of the receiver's advertised window (rwnd) and its own congestion window (cwnd). The congestion-control mechanisms of the TCP variant used in this case study (NewReno) are:

- **Slow start:** cwnd starts at one segment and grows by one segment for every ACK (it doubles every round-trip time) until it reaches the slow-start threshold (ssthresh) or a loss occurs [12].
- **Congestion avoidance:** above ssthresh, cwnd grows by about one segment per round-trip time.
- **Fast retransmit and fast recovery:** three duplicate ACKs signal a lost segment; the sender retransmits it immediately and halves cwnd. NewReno handles several losses from one window [13].
- **Retransmission timeout (RTO):** if no ACK arrives before the retransmission timer expires, the segment is retransmitted, cwnd falls to one segment, and the timer is doubled after every further unsuccessful attempt (exponential back-off) [14].

All of these mechanisms rest on one assumption: **a lost packet means that the network is congested**. This is true in wired networks, where losses come almost only from buffer overflow, but it is not true on wireless links.

3.1.2 Problems of TCP over Wireless and Mobile Links

Problem	Cause on a wireless/mobile link	Effect on TCP
Non-congestion packet loss	Fading, interference, collisions and low signal-to-noise ratio corrupt frames	TCP treats the loss as congestion, halves cwnd and lowers throughput although the network is not congested
Disconnection / out of range	The mobile node moves beyond the coverage area, behind an obstacle, or between access points	Retransmission timer expires repeatedly, cwnd falls to one segment and the transfer stalls
Slow recovery after reconnection	The RTO is doubled after every failed retransmission during the outage	The next retransmission can happen long after the link has returned, so the connection stays idle although the radio link is available
Variable delay and data rate	Link-layer retransmissions, rate adaptation and channel contention	RTT estimate becomes unreliable and spurious timeouts can occur
Low and time-varying bandwidth	Path loss grows with distance, so the MAC falls back to lower data rates	Throughput drops as the mobile node moves away from the access point
ACK traffic on a shared half-duplex channel	Wi-Fi ACK packets compete with data for the same channel	Reverse-path overhead and extra delay

3.1.3 Techniques Proposed to Improve TCP over Wireless

Technique	Idea
Link-layer ARQ (IEEE 802.11 retransmission)	Frames lost on the radio hop are retransmitted by the MAC layer, so TCP sees fewer losses. This is the only protection present in the simulation of Section 3.2.
Indirect TCP (I-TCP) [16]	The connection is split at the access point: a separate TCP connection (which can be tuned) is used for the wireless hop.
Snoop TCP [15]	An agent at the base station caches segments, retransmits locally when a loss is detected and suppresses duplicate ACKs so that the sender does not reduce its window.
M-TCP [17]	Split-connection scheme; when the mobile node is disconnected, the sender is frozen by advertising a zero window instead of timing out.
Freeze-TCP [18]	The mobile host advertises a zero window when a disconnection is about to occur, so the sender keeps its state and resumes at full rate when the link returns.

This case study uses standard TCP NewReno without any of these enhancements (apart from the Wi-Fi MAC retransmissions). The simulation therefore shows the "raw" behaviour of TCP when the wireless link degrades and disappears because of mobility.

3.2 Simulation Details [Code] and Results

3.2.1 Simulation Tool and Setup

The behaviour of TCP in a wireless/mobile environment was simulated with the **ns-3 network simulator (version 3.43)** running on Fedora Linux [19]. The program part3_tcp_wifi_mobile.cc (complete listing in Appendix A) follows the basic structure of the two reference ns-3 TCP-over-Wi-Fi examples, wifi-tcp.cc [20] and WIFI_TCP_simulation.cc [21], and extends it with node mobility, per-second throughput logging, a congestion-window trace, FlowMonitor statistics and PCAP tracing. A single mobile station (STA) sends TCP data to an access point (AP).

Parameter	Value
Simulator	ns-3.43 (C++), Fedora Linux
Topology	2 nodes: Node 0 = access point and TCP receiver (PacketSink); Node 1 = mobile station and TCP sender (BulkSend)
Wireless standard	IEEE 802.11n, infrastructure mode (one BSS, SSID "case-study-2")
Rate adaptation	IdealWifiManager (the data rate follows the received SNR)
Channel model	Default YansWifiChannel: log-distance path loss with constant-speed propagation delay
TCP	TcpNewReno, segment size 1448 bytes
Traffic	BulkSend (unlimited data) from t = 1 s to t = 60 s
Simulation time	60 s
Design 1	STA fixed at 5 m from the AP
Design 2	STA starts at 5 m, moves directly away from the AP at 4 m/s until t = 30 s (125 m), then returns at 4 m/s (ConstantVelocityMobilityModel)
Measurements	Throughput received at the AP every 1 s, congestion-window trace, FlowMonitor (packets, loss, delay), PCAP trace at the AP

3.2.2 Network Designs for the Demonstration

Both designs use the same program; only the station speed is different. **Design 1** (static station) is the baseline: it shows how TCP behaves on a good, stable Wi-Fi link. **Design 2** (mobile station) moves the station away from the access point, beyond the coverage area, and back again, so the wireless condition degrades and then recovers during the transfer.

Design 1 - Static station (baseline)



Design 2 - Mobile station moving out of range and back (shading = outward leg)

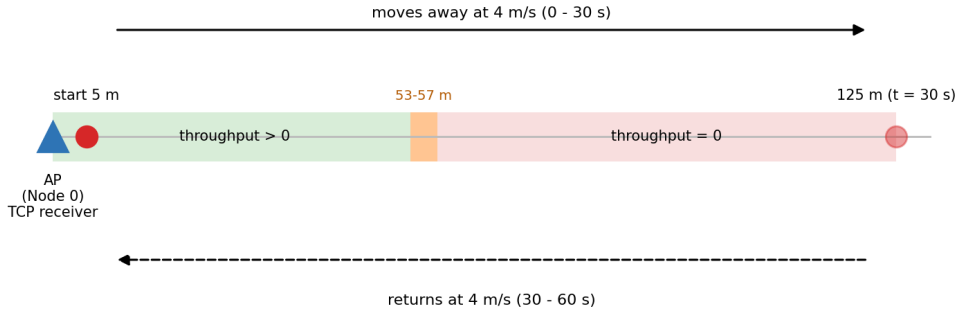


Figure 3.1: Network designs used for the TCP demonstration

The two runs are started with:

```
./ns3 run "scratch/part3_tcp_wifi_mobile --speed=0 --tag=design1"
./ns3 run "scratch/part3_tcp_wifi_mobile --speed=4 --tag=design2"
```

3.2.3 Sequence of Operations Supported by the Code Segments

Step	Operation	ns-3 code	Result

1	Select TCP variant and segment size	`Config::SetDefault(`
		`  "ns3::TcpL4Protocol::SocketType",`
		`  StringValue("ns3::TcpNewReno"));`
		`Config::SetDefault(`
		`  "ns3::TcpSocket::SegmentSize",`
		`  UintegerValue(1448));`
			TCP NewReno, 1448-byte segments

2	Create the nodes	`nodes.Create(2);  // 0 = AP, 1 = STA`	Two nodes

3	Configure Wi-Fi	`wifi.SetStandard(WIFI_STANDARD_80211n);`
		`wifi.SetRemoteStationManager(`
		`  "ns3::IdealWifiManager");`
			802.11n with SNR-based rate adaptation

4	Install MAC layers	`mac.SetType("ns3::StaWifiMac", ...);` `mac.SetType("ns3::ApWifiMac", ...);`	STA associates with the AP

5	Set up mobility	`staMob->SetPosition(Vector(startDist,0,0));`
		`staMob->SetVelocity(Vector(speed,0,0));`
		`Simulator::Schedule(Seconds(simTime/2.0),`
		`  &ConstantVelocityMobilityModel::`
		`  SetVelocity, staMob, Vector(-speed,0,0));`
			STA moves away and turns back at t = 30 s

6	Internet stack and IP addresses	`stack.Install(nodes);`
		`address.SetBase("10.1.1.0",`
		`  "255.255.255.0");`
			AP = 10.1.1.1, STA = 10.1.1.2

7	TCP applications	`BulkSendHelper bulk(`
		`  "ns3::TcpSocketFactory", ...);`
		`srcApp.Start(Seconds(1.0));`
		`PacketSinkHelper sinkHelper(...);`
			TCP flow from STA to AP

Step	Operation	ns-3 code	Result
8	Tracing	`Config::ConnectWithoutContext( "../CongestionWindow", ...); Simulator::Schedule(Seconds(2.0), &LogThroughput); phy.EnablePcap("part3-"+tag, apDev.Get(0));`	cwnd trace, throughput every second, PCAP file
9	Install FlowMonitor	`FlowMonitorHelper fmHelper; Ptr<FlowMonitor> monitor = fmHelper.InstallAll();`	Per-flow packet, loss and delay statistics
10	Run the simulation	`Simulator::Stop(Seconds(simTime)); Simulator::Run();`	60 s of simulated time
11	Print and save results	`monitor->GetFlowStats(); ... monitor->SerializeToXmlFile( tag+"-flowmon.xml", true, true);`	Console summary, .dat and .xml files

The complete program is listed in Appendix A. The data files written by the program (tag-throughput.dat and tag-cwnd.dat) were plotted with a short Python script (matplotlib).

3.2.4 Snapshot of the Execution

Figures 3.2 and 3.3 show the console output of the two runs: time, distance between STA and AP, and TCP throughput received at the AP in each one-second interval, followed by the FlowMonitor summary printed at the end of the simulation.

```
[saurav_gopinath_ek@fedora ns-3.43]$ ./ns3 run "scratch/part3_tcp_wifi_mobile --speed=0 --tag=design1"
[0/2] Re-checking globbed directories...
ninja: no work to do.
TCP variant: TcpNewReno | tag: design1 | simTime: 60 s | speed: 0 m/s | start distance: 5 m
time(s) dist(m) throughput(Mbps)
2.00    5.00    50.36
3.00    5.00    51.36
4.00    5.00    51.21
5.00    5.00    49.62
6.00    5.00    49.54
7.00    5.00    50.25
8.00    5.00    50.03
9.00    5.00    50.85
10.00   5.00    50.27
11.00   5.00    51.63
12.00   5.00    51.06
13.00   5.00    52.74
14.00   5.00    51.09
15.00   5.00    50.84
16.00   5.00    50.28
17.00   5.00    52.85
18.00   5.00    50.24
19.00   5.00    49.98
20.00   5.00    48.93
21.00   5.00    48.13
22.00   5.00    51.52
23.00   5.00    50.03
24.00   5.00    51.00
25.00   5.00    49.21
26.00   5.00    49.46
27.00   5.00    50.52
28.00   5.00    51.18
29.00   5.00    50.50
30.00   5.00    52.03
31.00   5.00    49.94
```

Figure 3.2(a): Design 1 (static station) – console output, throughput per second (t = 2–31 s)

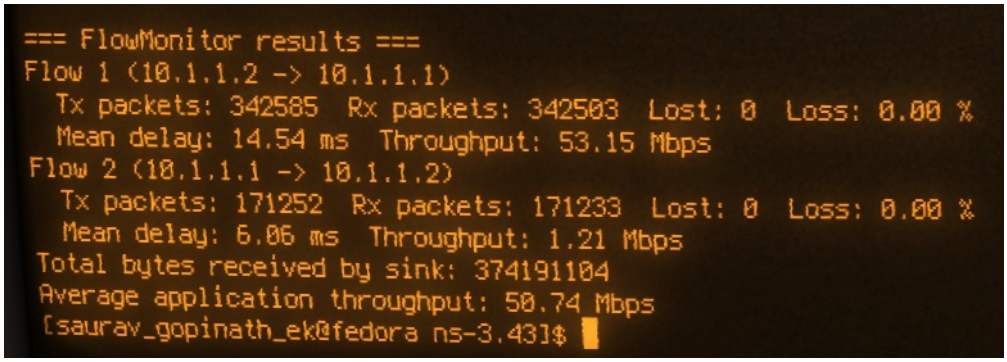


Figure 3.2(b): Design 1 – FlowMonitor summary at the end of the run

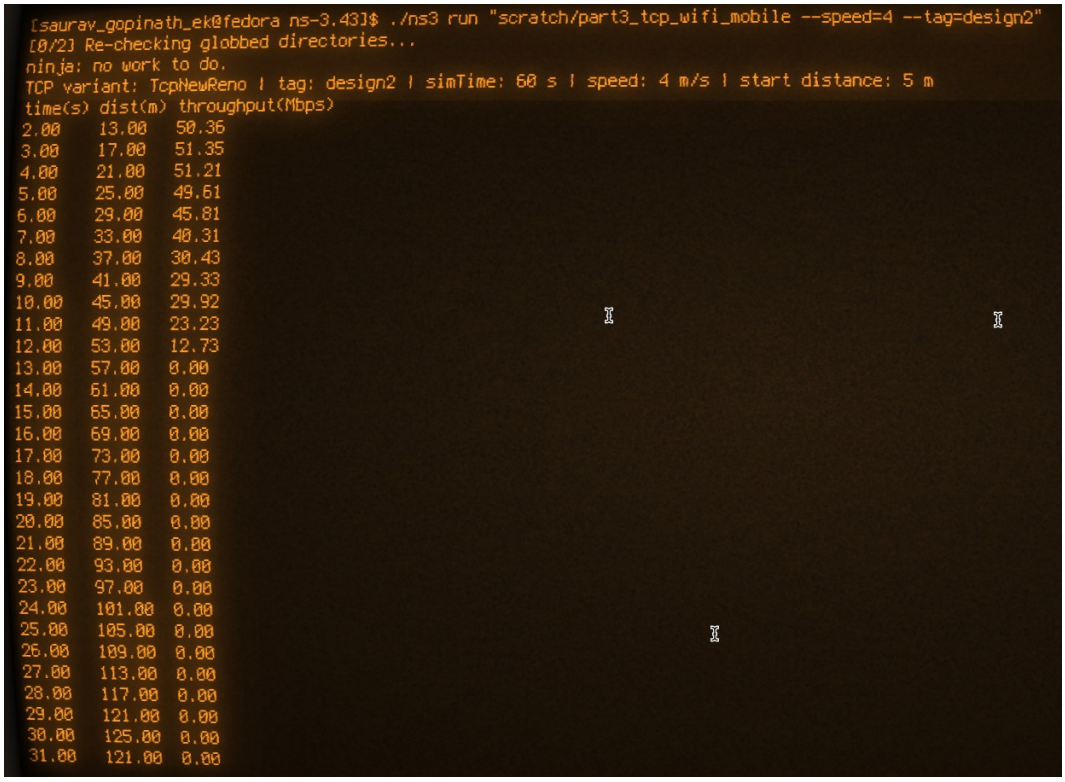


Figure 3.3(a): Design 2 (mobile station) – console output, throughput per second (t = 2–31 s)

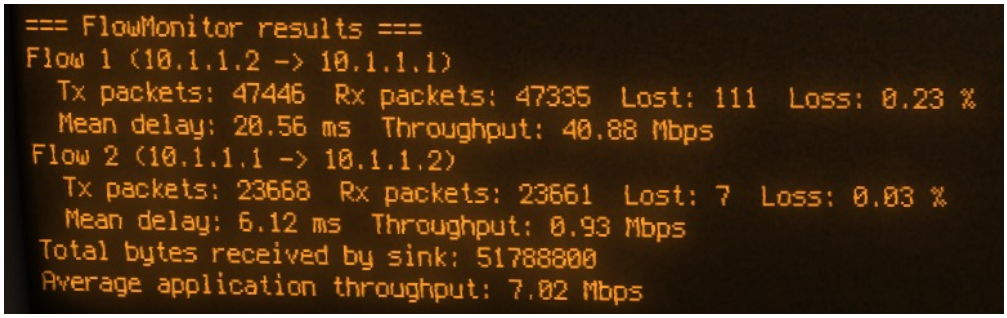


Figure 3.3(b): Design 2 – FlowMonitor summary at the end of the run

3.2.5 Results and Analysis

Design 1 (static station). The station stays 5 m from the access point. Throughput settles at about 50 Mbps and stays there for the whole run (Figure 3.2), and no data packet is lost.

Metric (Design 1)	Value
STA–AP distance	5 m (constant)
TCP throughput per second (t = 2–59 s)	48.13 – 52.85 Mbps
Average application throughput	50.74 Mbps
Data received by the sink	374,191,104 bytes (about 374.2 MB)
Data flow STA → AP: sent / received / lost	342,585 / 342,503 / 0 (0.00 %)
Data flow mean delay	14.54 ms
ACK flow AP → STA: sent / received / lost	171,252 / 171,233 / 0 (0.00 %)
ACK flow mean delay	6.06 ms

Design 2 (mobile station). The throughput follows the distance to the access point. It stays near 50 Mbps up to about 25 m, then decreases step by step, and drops to zero at 57 m (Table 3.1 and Figure 3.5). It never recovers during the rest of the run (Figure 3.4).

Table 3.1: Design 2 – TCP throughput versus distance on the outward leg (values from the simulator output)

Time (s)	Distance (m)	TCP throughput (Mbps)
2	13	50.36
3	17	51.35
4	21	51.21
5	25	49.61
6	29	45.81
7	33	40.31
8	37	30.43
9	41	29.33
10	45	29.92
11	49	23.23
12	53	12.73
13	57	0.00
14 – 59	61 m up to 125 m (turn at t = 30 s), then back to 9 m	0.00 in every interval (46 samples)

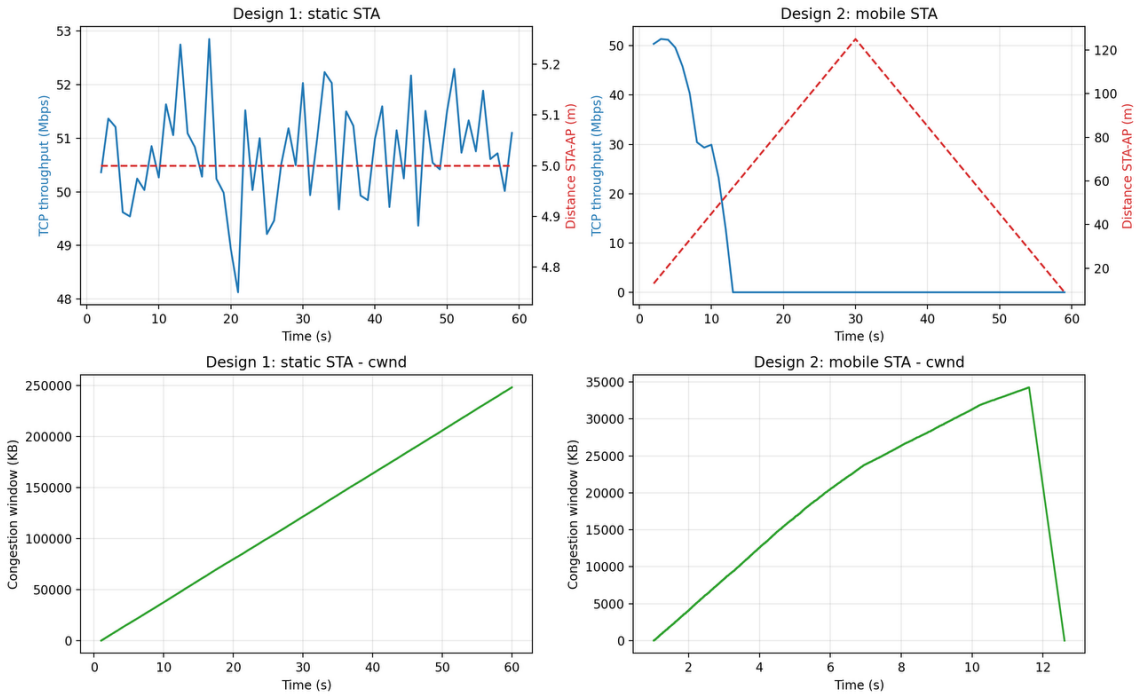


Figure 3.4: Throughput, STA–AP distance and TCP congestion window for Design 1 (left) and Design 2 (right). The cwnd of Design 2 has no further updates after about 12.6 s.

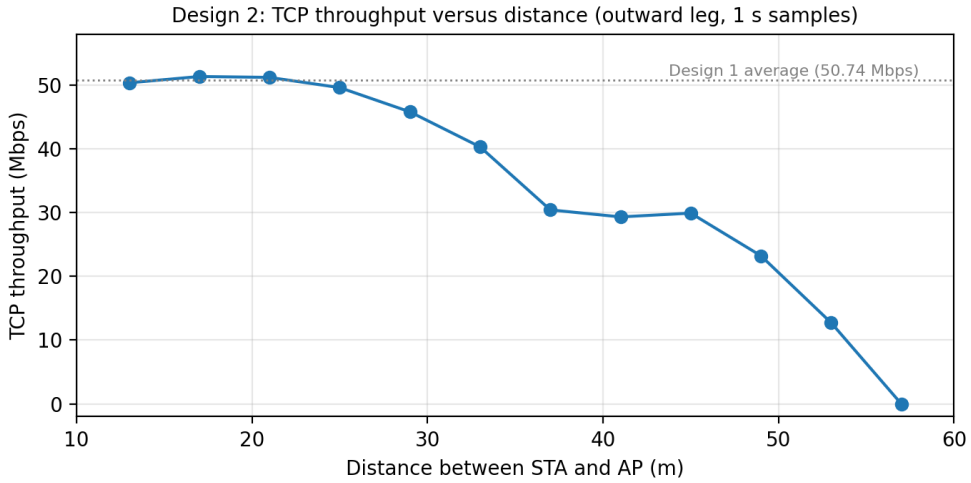


Figure 3.5: Design 2 – TCP throughput versus distance (outward leg)

Comparison of the two designs (FlowMonitor and sink statistics):

Metric	Design 1 (static)	Design 2 (mobile)
Average application throughput (1–60 s)	50.74 Mbps	7.02 Mbps
Data received by the sink	374.19 MB	51.79 MB
Seconds with zero throughput (of 58 samples)	0	47
Data flow: packets sent / received	342,585 / 342,503	47,446 / 47,335
Data flow: packets lost (loss ratio)	0 (0.00 %)	111 (0.23 %)
Data flow: mean delay	14.54 ms	20.56 ms
ACK flow: packets sent / received	171,252 / 171,233	23,668 / 23,661
ACK flow: packets lost (loss ratio)	0 (0.00 %)	7 (0.03 %)
ACK flow: mean delay	6.06 ms	6.12 ms
FlowMonitor data-flow throughput*	53.15 Mbps	40.88 Mbps

** FlowMonitor measures the IP-layer rate between the first transmitted and the last received packet. For Design 2 it therefore ignores the long idle period after the link was lost, so the application throughput measured at the sink is the meaningful figure.*

3.3 Observation from the Simulation

1. Stable link, stable TCP. With the station fixed at 5 m (Design 1), TCP reaches about 50 Mbps in the first measured interval (1–2 s) and holds it (48.1–52.9 Mbps) without any packet loss. The data flow has a mean delay of 14.5 ms because the bulk sender keeps the queues full; the ACK flow, which carries about one ACK for every two data segments (171,252 ACKs for 342,585 segments), has a mean delay of 6.1 ms.

2. cwnd keeps growing when nothing is lost. In Design 1 the congestion window rises almost linearly to about 248 MB (Figure 3.4). Because no loss ever occurs, TCP never leaves slow start and cwnd grows by one segment per ACK (about 171,000 ACKs × 1448 bytes ≈ 248 MB). The window is therefore not what limits the throughput; the capacity of the 802.11n link (and the socket buffers) does.

3. Throughput falls with distance even without TCP losses. In Design 2 the throughput is about 50 Mbps up to 25 m and then falls in steps: 45.8 Mbps at 29 m, 30.4 Mbps at 37 m, 23.2 Mbps at 49 m and 12.7 Mbps at 53 m. The path loss grows with distance and the received SNR drops, which is what the SNR-based rate adaptation (IdealWifiManager) answers by selecting lower data rates. This is a wireless-link effect that TCP can only follow, not prevent.

4. Leaving the coverage area stops the transfer. Between 53 m and 57 m (t ≈ 12–13 s) the throughput falls to zero. The cwnd trace shows the window collapsing to about one segment at t ≈ 12.6 s, which is consistent with an expired retransmission timer (RTO). TCP cannot tell a disconnection from congestion, so it treats the outage as heavy congestion. The cut-off distance of about 51–57 m is consistent with the default ns-3 receiver model: with the log-distance model (exponent 3, 46.68 dB at 1 m) and a transmit power of 16.02 dBm, the received power equals the default preamble-detection limit of −82 dBm at about 51 m (calculated value).

5. TCP does not resume when the link returns. The station is back inside the coverage area from about t = 48–49 s (49 m at 49 s) and reaches 9 m at 59 s, closer than at the start where 50 Mbps was achieved, yet the throughput is still 0 Mbps. In total 47 consecutive one-second samples (t = 13–59 s) show zero throughput. This is consistent with the exponential back-off of the TCP retransmission timer: after the first timeout at about 12.6 s the timer doubles after every failed attempt, so retransmission attempts are expected at roughly 14.6, 18.6, 26.6, 42.6 and 74.6 s (calculated from the observed first timeout), and none of them falls in the period in which the station was in range. The time the station needs to re-associate with the access point after hearing its beacons again may add to the delay. This is the third problem listed in Section 3.1.2: the connection stays idle although the radio link is available.

6. The loss ratio hides the real damage. The packet-loss ratio of Design 2 is only 0.23 % (111 of 47,446 data packets), because TCP stops sending once its window and timer collapse. The real effect is that only 13.8 % of the data was delivered (51.79 MB against 374.19 MB), and the average throughput fell from 50.74 Mbps to 7.02 Mbps, a reduction of 86 %.

7. Delay increases when the link is weaker. The mean delay of the data flow rose from 14.54 ms to 20.56 ms (about 6 ms more) because frames sent at lower data rates occupy

the channel longer. The delay of the small ACK packets did not change (6.06 ms against 6.12 ms).

8. Relation to Part 1 and Part 2. Mobile IP (Part 1) lets the mobile node keep its IP address, and MANET routing (Part 2) keeps routes available, but neither can prevent this transport-layer problem: the TCP timer behaviour of a standard sender remains unchanged. Mechanisms such as Snoop, M-TCP and Freeze-TCP (Section 3.1.3) are designed for exactly this situation.

9. Scope and limits. Each design was simulated once (the ns-3 run is deterministic) with a single TCP variant (NewReno), two nodes, no competing traffic and the default idealised propagation model. Possible extensions are a comparison of other TCP variants (for example Westwood or Cubic) and a shorter outage to measure the recovery time of TCP.

Conclusion. The simulation shows that standard TCP performs very well on a stable wireless link, but that mobility changes the picture completely: weaker signal means lower throughput, loss of coverage stops the transfer, and the exponential back-off of the retransmission timer delays recovery long after the link is available again. Better wireless-aware TCP mechanisms are therefore needed in mobile environments.

SECTION 4: REFERENCES

Part 1 – Mobile IP

[1] C. Perkins (Ed.), "IP Mobility Support for IPv4," RFC 3344, IETF, August 2002.

[2] C. Perkins, "IP Encapsulation within IP," RFC 2003, IETF, October 1996.

[3] Scapy documentation, scapy.layers.mobileip: https://scapy.readthedocs.io/en/latest/api/scapy.layers.mobileip.html

[4] Proxy Mobile IPv6 for cloud: https://repozitorium.omikk.bme.hu/items/c8b8f272-26ee-4775-a8d3-c7caa9730086

[5] Wireshark network protocol analyzer: https://www.wireshark.org

Part 2 – MANET routing (OLSR)

[6] T. Clausen, P. Jacquet (Eds.), "Optimized Link State Routing Protocol (OLSR)," RFC 3626, IETF, October 2003.

[7] MPR Selection in OLSR protocol – MATLAB Central File Exchange: https://in.mathworks.com/matlabcentral/fileexchange/71079-mpr-selection-in-olsr-protocol

[8] MSH19, MPR_Selection – algorithm for MPR nodes selection in OLSR protocol (GitHub): https://github.com/MSH19/MPR_Selection

[9] MathWorks, Performance Evaluation of TDMA-Based MANET: https://in.mathworks.com/help/wireless-network/ug/performance-evaluation-of-tdma-based-manet.html

[10] MathWorks, Mobile Ad Hoc Network Modeling: https://in.mathworks.com/help/wireless-network/mobile-ad-hoc-networks.html

Part 3 – TCP over wireless/mobile environment

[11] W. Eddy (Ed.), "Transmission Control Protocol (TCP)," RFC 9293, IETF, August 2022.

[12] M. Allman, V. Paxson, E. Blanton, "TCP Congestion Control," RFC 5681, IETF, September 2009.

[13] T. Henderson, S. Floyd, A. Gurtov, Y. Nishida, "The NewReno Modification to TCP's Fast Recovery Algorithm," RFC 6582, IETF, April 2012.

[14] V. Paxson, M. Allman, J. Chu, M. Sargent, "Computing TCP's Retransmission Timer," RFC 6298, IETF, June 2011.

[15] H. Balakrishnan, S. Seshan, E. Amir, R. H. Katz, "Improving TCP/IP Performance over Wireless Networks," ACM MobiCom, 1995.

[16] A. Bakre, B. R. Badrinath, "I-TCP: Indirect TCP for Mobile Hosts," IEEE ICDCS, 1995.

[17] K. Brown, S. Singh, "M-TCP: TCP for Mobile Cellular Networks," ACM SIGCOMM Computer Communication Review, vol. 27, no. 5, 1997.

[18] T. Goff, J. Moronski, D. S. Phatak, V. Gupta, "Freeze-TCP: A True End-to-End TCP Enhancement Mechanism for Mobile Environments," IEEE INFOCOM, 2000.

[19] ns-3 Network Simulator, documentation and tutorial (ns-3.43): https://www.nsnam.org/documentation/

[20] wifi-tcp.cc, ns-3 TCP over Wi-Fi example: https://github.com/Lunodzo/ns3-simulations/blob/master/wifi-tcp.cc

[21] WIFI_TCP_simulation.cc, ns-3 TCP over Wi-Fi example: https://github.com/libremente/NS3-simulation/blob/master/WIFI_TCP_simulation.cc

APPENDIX A: COMPLETE SOURCE CODE OF part3_tcp_wifi_mobile.cc

ns-3 program used for Section 3 (written for ns-3.36 or newer; run from the ns-3 directory as shown in Section 3.2.2).

```
/*
 * part3_tcp_wifi_mobile.cc
 * 23CSE343 Wireless & Mobile Networks - Case Study 2, Part 3 (Team 10)
 *
 * TCP over an 802.11n Wi-Fi link with a mobile station.
 *   Node 0 = Access Point (TCP receiver / PacketSink)
 *   Node 1 = Mobile STA   (TCP sender / BulkSend)
 *
 * Written for ns-3.36 or newer (uses WIFI STANDARD 80211n).
 *
 * Design 1 (static STA):
 *   ./ns3 run "scratch/part3_tcp_wifi_mobile --speed=0 --tag=design1"
 * Design 2 (STA moves away, turns round at half time, comes back):
 *   ./ns3 run "scratch/part3_tcp_wifi_mobile --speed=4 --tag=design2"
 *
 * Output files (written to the directory you run from):
 *   <tag>-throughput.dat   time  distance  throughput(Mbps)   (every 1 s)
 *   <tag>-cwnd.dat         time  cwnd(bytes)                  (every change)
 *   <tag>-flowmon.xml      FlowMonitor statistics
 *   part3-<tag>-0-0.pcap   packet capture at the AP (open in Wireshark)
 */

#include "ns3/applications-module.h"
#include "ns3/core-module.h"
#include "ns3/flow-monitor-module.h"
#include "ns3/internet-module.h"
#include "ns3/mobility-module.h"
#include "ns3/network-module.h"
#include "ns3/wifi-module.h"

#include <fstream>
#include <iomanip>
#include <iostream>

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("Part3TcpWifiMobile");

static Ptr<PacketSink> g_sink;
static Ptr<MobilityModel> g_apMob;
static Ptr<MobilityModel> g_staMob;
static uint64_t g_lastRx = 0;
static std::ofstream g_thrFile;
static std::ofstream g_cwndFile;

// Called every second: throughput received at the AP + current distance
static void
LogThroughput()
{
    uint64_t total = g_sink->GetTotalRx();
    double mbps = (total - g_lastRx) * 8.0 / 1e6; // bytes in the last 1 s -> Mbps
    g_lastRx = total;
    double dist = g_staMob->GetDistanceFrom(g_apMob);
    double t = Simulator::Now().GetSeconds();

    std::cout << std::fixed << std::setprecision(2) << t << "\t" << dist << "\t" << mbps
              << std::endl;
    g_thrFile << std::fixed << std::setprecision(3) << t << "\t" << dist << "\t" << mbps
              << std::endl;

    Simulator::Schedule(Seconds(1.0), &LogThroughput);
}

// Congestion-window trace of the sender's TCP socket
```

```
static void
CwndChange(uint32_t oldCwnd, uint32_t newCwnd)
{
    g_cwndFile << std::fixed << std::setprecision(4) << Simulator::Now().GetSeconds() << "\t"
               << newCwnd << std::endl;
}


// The TCP socket only exists after BulkSend starts, so connect the trace afterwards
static void
ConnectCwndTrace()
{

Config::ConnectWithoutContext("/NodeList/1/$ns3::TcpL4Protocol/SocketList/0/CongestionWindow",
                                  MakeCallback(&CwndChange));
}


int
main(int argc, char* argv[])
{
    std::string tcpVariant = "TcpNewReno";
    std::string tag = "design1";
    double simTime = 60.0;   // seconds
    double speed = 0.0;      // m/s (0 = static STA)
    double startDist = 5.0;  // initial distance STA <-> AP in metres
    uint32 t segSize = 1448; // TCP segment size in bytes
    bool pcap = true;


    CommandLine cmd(__FILE__);
    cmd.AddValue("tcp", "TCP variant: TcpNewReno, TcpWestwood, TcpCubic, TcpLinuxReno ...",
tcpVariant);
    cmd.AddValue("tag", "Label used in output file names", tag);
    cmd.AddValue("simTime", "Simulation time in seconds", simTime);
    cmd.AddValue("speed", "STA speed in m/s (0 = static)", speed);
    cmd.AddValue("startDist", "Initial STA-AP distance in metres", startDist);
    cmd.AddValue("pcap", "Enable PCAP tracing at the AP", pcap);
    cmd.Parse(argc, argv);


    Config::SetDefault("ns3::TcpL4Protocol::SocketType", StringValue("ns3::" + tcpVariant));
    Config::SetDefault("ns3::TcpSocket::SegmentSize", UintegerValue(segSize));


    // ---- Nodes: 0 = AP, 1 = STA ----
    NodeContainer nodes;
    nodes.Create(2);
    Ptr<Node> apNode = nodes.Get(0);
    Ptr<Node> staNode = nodes.Get(1);


    // ---- Wi-Fi (802.11n, log-distance path loss) ----
    YansWifiChannelHelper channel = YansWifiChannelHelper::Default();
    YansWifiPhyHelper phy;
    phy.SetChannel(channel.Create());


    WifiHelper wifi;
    wifi.SetStandard(WIFI STANDARD 80211n);
    wifi.SetRemoteStationManager("ns3::IdealWifiManager"); // rate adapts to SNR


    WifiMacHelper mac;
    Ssid ssid = Ssid("case-study-2");
    mac.SetType("ns3::StaWifiMac", "Ssid", SsidValue(ssid), "ActiveProbing",
BooleanValue(false));
    NetDeviceContainer staDev = wifi.Install(phy, mac, staNode);
    mac.SetType("ns3::ApWifiMac", "Ssid", SsidValue(ssid));
    NetDeviceContainer apDev = wifi.Install(phy, mac, apNode);


    // ---- Mobility: AP fixed at origin, STA moves along the x axis ----
    MobilityHelper mobility;
    mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
    mobility.Install(apNode);
    mobility.SetMobilityModel("ns3::ConstantVelocityMobilityModel");
    mobility.Install(staNode);


    Ptr<ConstantPositionMobilityModel> apMob = apNode-
>GetObject<ConstantPositionMobilityModel>();
    apMob->SetPosition(Vector(0.0, 0.0, 0.0));
    Ptr<ConstantVelocityMobilityModel> staMob = staNode-
>GetObject<ConstantVelocityMobilityModel>();
```

```
    staMob->SetPosition(Vector(startDist, 0.0, 0.0));
    staMob->SetVelocity(Vector(speed, 0.0, 0.0));
    if (speed > 0)
    {
        // turn round at half time and come back towards the AP
        Simulator::Schedule(Seconds(simTime / 2.0),
                            &ConstantVelocityMobilityModel::SetVelocity,
                            staMob,
                            Vector(-speed, 0.0, 0.0));
    }
    g_apMob = apMob;
    g_staMob = staMob;


    // ---- Internet stack + IP addresses ----
    InternetStackHelper stack;
    stack.Install(nodes);
    Ipv4AddressHelper address;
    address.SetBase("10.1.1.0", "255.255.255.0");
    Ipv4InterfaceContainer apIf = address.Assign(apDev);
    address.Assign(staDev);


    // ---- TCP applications: STA sends (BulkSend), AP receives (PacketSink) ----
    uint16_t port = 9;
    PacketSinkHelper sinkHelper("ns3::TcpSocketFactory",
                                InetSocketAddress(Ipv4Address::GetAny(), port));
    ApplicationContainer sinkApp = sinkHelper.Install(apNode);
    sinkApp.Start(Seconds(0.0));
    sinkApp.Stop(Seconds(simTime));
    g_sink = DynamicCast<PacketSink>(sinkApp.Get(0));


    BulkSendHelper bulk("ns3::TcpSocketFactory", InetSocketAddress(apIf.GetAddress(0), port));
    bulk.SetAttribute("MaxBytes", UintegerValue(0)); // send until the end
    ApplicationContainer srcApp = bulk.Install(staNode);
    srcApp.Start(Seconds(1.0));
    srcApp.Stop(Seconds(simTime));


    // ---- Tracing / statistics ----
    g_thrFile.open(tag + "-throughput.dat");
    g_cwndFile.open(tag + "-cwnd.dat");
    Simulator::Schedule(Seconds(1.01), &ConnectCwndTrace);
    Simulator::Schedule(Seconds(2.0), &LogThroughput); // first sample covers 1 s..2 s


    if (pcap)
    {
        phy.EnablePcap("part3-" + tag, apDev.Get(0));
    }


    FlowMonitorHelper fmHelper;
    Ptr<FlowMonitor> monitor = fmHelper.InstallAll();


    std::cout << "TCP variant: " << tcpVariant << " | tag: " << tag << " | simTime: " <<
simTime
              << " s | speed: " << speed << " m/s | start distance: " << startDist << " m"
              << std::endl;
    std::cout << "time(s)\tdist(m)\tthroughput(Mbps)" << std::endl;


    Simulator::Stop(Seconds(simTime));
    Simulator::Run();


    // ---- FlowMonitor summary ----
    monitor->CheckForLostPackets();
    Ptr<Ipv4FlowClassifier> classifier =
DynamicCast<Ipv4FlowClassifier>(fmHelper.GetClassifier());
    std::cout << "\n=== FlowMonitor results ===" << std::endl;
    for (const auto& kv : monitor->GetFlowStats())
    {
        Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(kv.first);
        const FlowMonitor::FlowStats& s = kv.second;
        double dur = s.timeLastRxPacket.GetSeconds() - s.timeFirstTxPacket.GetSeconds();
        double thr = (dur > 0) ? s.rxBytes * 8.0 / dur / 1e6 : 0.0;
        double delayMs = (s.rxPackets > 0) ? s.delaySum.GetSeconds() * 1000.0 / s.rxPackets :
0.0;
        double lossPct = (s.txPackets > 0) ? 100.0 * s.lostPackets / s.txPackets : 0.0;
        std::cout << "Flow " << kv.first << " (" << t.sourceAddress << " -> "
                  << t.destinationAddress << ")" << std::endl;
        std::cout << "  Tx packets: " << s.txPackets << "  Rx packets: " << s.rxPackets
```

```
                  << "  Lost: " << s.lostPackets << "  Loss: " << std::fixed
                  << std::setprecision(2) << lossPct << " %" << std::endl;
        std::cout << "  Mean delay: " << delayMs << " ms  Throughput: " << thr << " Mbps"
                  << std::endl;
    }
    std::cout << "Total bytes received by sink: " << g_sink->GetTotalRx() << std::endl;
    std::cout << "Average application throughput: "
              << g_sink->GetTotalRx() * 8.0 / (simTime - 1.0) / 1e6 << " Mbps" << std::endl;


    monitor->SerializeToXmlFile(tag + "-flowmon.xml", true, true);


    g_thrFile.close();
    g_cwndFile.close();
    Simulator::Destroy();
    return 0;
}
```